

Programare Orientată Pe Obiecte

Curs 14: Exemplu Rezolvare Examen

Ianuarie 2026

Structura examenului

- Problema 1: ce se afișează la rularea unui cod dat
- Problema 2: întrebări de tipul True / False
- Problemele 3, 4, 5 : teorie (definire si explicații concepte studiate la curs)
- Problema 6: să se scrie codul pentru o aplicație JAVA
 - Exemplu: Scrieți codul JAVA complet pentru a simula jucarea unor meciuri în sporturile de echipă (VOLEI, FOTBAL, BASCHET, POLO) conform specificațiilor date.
 - Identificați și definiți clasele (cu metode, attribute) relevante pentru rezolvarea cerințelor.
 - Utilizați în cod următoarele: mostenire, clasă abstractă, polimorfism, interfață, variabilă statică, apel al constructorului din clasa părinte
 - Codul va cuprinde și o clasă cu metoda main() în care se vor crea mai mulți jucători, se vor crea echipe și meciuri.
 - Desenați diagrama UML de clase pentru aplicație.

Întrebarea 1:

Ce este cuvântul cheie `final` în Java?

Explicați utilizările sale (variabile, metode, clase).

Dați exemplu de cod Java pentru utilizările lui `final`.

Întrebarea 1:

Ce este cuvântul cheie `final` în Java?

Explicați utilizările sale (variabile, metode, clase).

Dați exemplu de cod Java pentru utilizările lui `final`.

Rezolvare:

Cuvântul cheie `final` poate fi folosit pentru:

- Variabile: valoarea nu poate fi modificată după inițializare.
- Metode: metoda nu poate fi suprascrisă în clasele derivate.
- Clase: clasa nu poate fi extinsă.

Exemplu:

```
final class Constants {  
    static final double PI = 3.14;  
}
```

Întrebarea 2:

- a) Cum se realizează gestiunea excepțiilor în Java?
- b) Dați un exemplu în care să aveți blocurile `try-catch-finally`.
- c) Explicați situația în care există blocuri `catch multiple` și dați exemplu de cod cu blocuri `catch multiple`.

Întrebarea 2: Rezolvare a)

- a) Cum se realizează gestiunea excepțiilor în Java?
- b) Dați un exemplu în care să aveți blocurile `try-catch-finally`.
- c) Explicați situația în care există blocuri `catch` multiple și dați exemplu de cod cu blocuri `catch` multiple.
 - La aruncarea unei excepții se începe executarea blocului `catch`.
 - Blocul `catch` are un parametru: obiectul excepție aruncat este transmis ca parametru al blocului `catch`
 - Un bloc `catch` este o porțiune de cod separată care se execută atunci când un program întâlnește și execută o instrucțiune `throw` în blocul `try` precedent]item Execuția blocului `catch` se numește interceptarea/" prinderea" excepției, sau tratarea excepției

Întrebarea 2: Rezolvare a) - continuare

- a) Cum se realizează gestiunea excepțiilor în Java?
- b) Dați un exemplu în care să aveți blocurile `try-catch-finally`.
- c) Explicați situația în care există blocuri `catch` multiple și dați exemplu de cod cu blocuri `catch` multiple.

```
try{  
    //cod care arunca exceptie  
}  
catch(Exception e) { CodDeTratareAExceptiei }
```

- Identificatorul **e** din blocul **catch** de deasupra se numește **parametru al blocului catch** și îndeplinește două roluri:
 - ❶ Specifică tipul de obiect excepție aruncat pe care blocul `catch` îl poate intercepta (d.e., mai sus este un obiect de clasa `Exception`)
 - ❷ Oferă un nume (pentru obiectul care este interceptat) care să fie folosit în blocul `catch`
 - ❸ Observație: adesea se folosește identificatorul **e** prin convenție, dar se poate folosi orice identificator care nu este cuvânt cheie.

Întrebarea 2: Rezolvare b): exemplu cu try-catch-finally

- Cum se realizează gestiunea excepțiilor în Java?
- Dați un exemplu în care să aveți blocurile try-catch-finally.
- Explicați situația în care există blocuri catch multiple și dați exemplu de cod cu blocuri catch multiple.

```
public class TestMultipleCatchBlock {  
    public static void main(String args[]){  
        try{  
            int a[]=new int[5];  
            a[5]=30/0;  
        } catch(ArithmeticException e){System.out.print("task1 done");  
        } catch(ArrayIndexOutOfBoundsException e){  
            System.out.println("task2 done");  
        } catch(Exception e){System.out.println("common task completed");}  
        finally{Sysout("finally")}  
        System.out.println("rest of the code...");  
    } }
```

- Nu e nevoie să apară o excepție pentru ca să se execute blocul finally.
- Blocul finally se execută întotdeauna.
- Finally nu se execută dacă apare o eroare care cauzează System.exit (out of memory) RAR !

Întrebarea 2: Rezolvare c): blocuri catch multiple:

- a) Cum se realizează gestiunea excepțiilor în Java?
 - b) Dați un exemplu în care să aveți blocurile `try-catch-finally`.
 - c) Explicați situația în care există blocuri `catch` multiple și dați exemplu de cod cu blocuri `catch` multiple.
- Un bloc `try` poate arunca potențial orice număr de valori excepție, iar acestea pot fi de tipuri diferite
 - În oricare execuție a unui bloc `try`, poate fi aruncată cel mult o excepție (de vreme ce instrucțiunea `throw` termină execuția blocului `try`)
 - La execuții diferite ale blocului `try` pot fi aruncate valori diferite
 - Fiecare bloc `catch` poate intercepta valorile de tipul de clasă excepție, date în antetul blocului `catch`
 - Se pot intercepta tipuri diferite de excepții punând mai multe blocuri `catch` după un bloc `try`
 - Se pot pune oricâte blocuri `catch`, dar în ordinea corectă

Întrebarea 3:

Ce este încapsularea în Java?

Explicați cum se realizează și oferiți un exemplu.

Întrebarea 3:

Ce este încapsularea în Java?

Explicați cum se realizează și oferiți un exemplu.

Rezolvare: Încapsularea este procesul de ascundere a detaliilor interne ale unui obiect și expunerea unei interfețe publice. Se realizează folosind modificatori de acces (private) și metode getter și setter. Exemplu:

```
class Student {  
    private String name;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Întrebarea 4:

Explicați mecanismul de definire și instanțiere a unui `Thread` în JAVA. Dați exemplu de cod JAVA în care să creați cel puțin două Thread-uri diferite.

Întrebarea 4:

Explicați mecanismul de definire și instanțiere a unui Thread în JAVA. Dați exemplu de cod JAVA în care să creați cel puțin două Thread-uri diferite.

Un Thread reprezintă un fir de execuție. Se creează prin extinderea clasei Thread sau prin implementarea interfeței Runnable.

Exemplu:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Firul ruleaza");
    }
}

class MyRunnable implements Runnable {
    public void run() { System.out.println("Firul este in executie"); }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();

        MyRunnable r = new MyRunnable();
        Thread t1 = new Thread(r);
        t1.start();
    }
}
```

5: Ce afișează + explicații

```
abstract class Vehicul {
    String combustibil = "neidentificat";
    String marca;
    Vehicul(String marca, String combustibil) {
        System.out.println("Vehicul: inainte de apelul draw()");
        this.marca = marca;
        draw();
        this.combustibil = combustibil;    }
    abstract void draw();
}

class Masina extends Vehicul {
    private int viteza = 0;
    private static String tipTransmisie = "manuala";
    Masina(String marca, String c, int viteza) {
        super(marca, c);
        this.viteza = viteza;
        System.out.println("Constructor: marca=" + marca + ", viteza=" + viteza + ",
            tipTransmisie = " + tipTransmisie + ", combustibil=" + combustibil);    }
    void draw() {
        System.out.println("Draw: marca = " + marca + ", viteza = " + viteza + ",
            tipTransmisie = " + tipTransmisie + ", combustibil = " + combustibil);
    }
}

public class Main {
    public static void main(String[] args) {
        new Masina("BMW", "benzina", 120);
    }
}
```

5: Ce afișează + explicații

Output-ul va fi:

```
Vehicul: înainte de apelul draw()
Draw: marca = BMW, viteza = 0, tipTransmisie = manuala, combustibil =
      neidentificat
Constructor: marca = BMW, viteza = 120, tipTransmisie = manuala,
            combustibil = benzina
```

Explicație:

- Constructorul clasei părinte este apelat înainte de cel al clasei derivate.
- Metoda `display` este suprascrisă, deci cea din `Child` este apelată datorită polimorfismului.
- Variabilele statice sunt accesate în funcție de clasă în care sunt definite.

Întrebarea 6:

Scrieți codul Java pentru o aplicație care simulează gestionarea dispozitivelor smart dintr-o locuință, considerând următoarele:

- a) Fiecare Dispozitiv este caracterizat de nume (String), stare (pornit/oprit boolean), consumEnergetic (double, consumul în wați), și un Id unic (generat automat). Fiecare Dispozitiv are o metodă de `control()` folosită pentru a porni / opri dispozitivul, o metodă care afișează datele despre dispozitiv și o metodă de resetare a parametrilor dispozitivului.
- b) Dispozitivele pot fi Televizor (canal curent, volum), Frigider (temperatura curentă, capacitatea totală în litri), Bec (intensitate luminoasă, culoarea luminii[alb cald, alb rece, neutru]).
- c) Utilizați pentru implementarea dispozitivelor clasă abstractă și moștenire.
- d) Creați o interfață `SmartControl` cu metoda `activareModEco()` pentru a reduce consumul energetic. La activarea modului eco la un televizor i se reduce consumul energetic cu 20%, la un frigider crește temperatura cu 2 grade și se reduce consumul energetic cu 10%, iar la un bec intensitatea scade cu 50 iar consumul energetic se reduce cu 30%.

Scriveți codul Java pentru o aplicație care simulează gestionarea dispozitivelor smart dintr-o locuință, considerând următoarele:

- e) Adăugați funcționalitatea de sortare a dispozitivelor după status (pornit/oprit) și consum energetic. Dispozitivele pornite trebuie să fie afișate primele, în ordinea descrescătoare a consumului energetic, urmate de dispozitivele oprite, tot în ordinea descrescătoare a consumului energetic.
 - f) Implementați o metodă care calculează consumul total al dispozitivelor pornite. Metoda trebuie să fie statică și să fie adăugată într-o clasă separată numită `UtilitareCasaSmart`. Această metodă va parcurge lista de dispozitive și va calcula suma consumului energetic doar pentru dispozitivele pornite.
 - g) Includeți și o clasă principală în care creați o listă de dispozitive (de diferite tipuri). Afișați informațiile fiecărui dispozitiv. Simulați pornirea/oprirea dispozitivelor, activarea modului eco pentru toate dispozitivele, sortarea lor după stare și calcularea consumului total.
 - h) Desenați diagrama UML de clase.
- *** Se presupune că metodele setter și getter pentru atributele din toate clasele sunt definite (nu e nevoie să le implementați).

Întrebare 6: Rezolvare

```
interface SmartControl {
    void activareModEco();
    void resetare();
}

abstract class Dispozitiv implements Comparable<Dispozitiv> {
    protected String nume;
    protected boolean stare; // true = pornit, false = oprit
    protected double consumEnergetic;
    protected final int id; // ID unic, generat automat
    private static int contorId = 0; // contor static pt generare ID
    private static int totalDispozitive = 0;
    public Dispozitiv(String nume, double consumEnergetic) {
        this.id = ++contorId; // ID unic generat automat
        this.nume = nume;
        this.consumEnergetic = consumEnergetic;
        this.stare = false;
        totalDispozitive++;
    }
    public abstract void control();
}
```

Întrebare 6: Rezolva

```
public void afiseazaInfo() {
    System.out.println("ID: " + id +
        ", Dispozitiv: " + nume +
        ", Stare: " + (stare ? "Pornit" : "Oprit") +
        ", Consum: " + consumEnergetic + "W");
}

@Override
public String toString() {
    return "ID: " + id + ", " + nume +
        " [" + (stare ? "Pornit" : "Oprit") +
        ", " + consumEnergetic + "W]";
}

public static void afiseazaTotalDispozitive() {
    System.out.println("Total dispozitive: " + totalDispozitive);
}

public boolean isPornit() { return stare; }

@Override
public int compareTo(Dispozitiv d) {
    // Sortare descrescatoare dupa consum energetic
    return Double.compare(d.getConsumEnergetic(), this.
        getConsumEnergetic());
}
```

Întrebare 6: Rezolvare

```
class Televizor extends Dispozitiv implements SmartControl {
    private int canal;
    private int volum;
    public Televizor(String nume, double consum, int canal, int volum) {
        super(nume, consum);
        this.canal = canal;
        this.volum = volum;
    }
    @Override
    public void control() {
        stare = !stare;
        System.out.println(nume + " a fost " + (stare ? "pornit" : "oprit"));
    }
    @Override
    public void activareModEco() {
        consumEnergetic *= 0.8; // reducere consum cu 20%
        System.out.println(nume + " in mod Eco. Consum: " + consumEnergetic + "W");
    }
    @Override
    public void resetare() {
        canal = 1;
        volum = 10;
        System.out.println(nume + " resetat.");
    }
    @Override
    public void afiseazaInfo() {
        super.afiseazaInfo();
        System.out.println(" Canal: " + canal + ", Volum: " + volum);
    }
}
```

Întrebare 6: Rezolvare

```
class Frigider extends Dispozitiv implements SmartControl {
    private double temperatura;
    private int capacitate; // in litri
    public Frigider(String nume, double consum, double temperatura, int capacitate) {
        super(nume, consum);
        this.temperatura = temperatura;
        this.capacitate = capacitate;
    }
    @Override
    public void control() {
        stare = !stare;
        System.out.println(nume + " a fost " + (stare ? "pornit" : "oprit"));
    }
    @Override
    public void activareModEco() {
        temperatura += 2; // creste temperatura cu 2 grade
        consumEnergetic *= 0.9; // reduce consum cu 10%
        System.out.println(nume + " in mod Eco. Temperatura: " + temperatura +
            ", Consum: " + consumEnergetic + "W");
    }
    @Override
    public void resetare() {
        temperatura = 4;
        System.out.println(nume + " resetat. Temperatura: " + temperatura);
    }
    @Override
    public void afiseazaInfo() {
        super.afiseazaInfo();
        System.out.println(" Temperatura: " + temperatura + ", Capacitate: " + capacitate +
            "L");
    }
}
```

Întrebare 6: Rezolvare

```
class Bec extends Dispozitiv implements SmartControl {
    private int intensitate; // 0-100
    private String culoare; // "alb cald", "alb rece", "neutru"
    public Bec(String nume, double consum, int intensitate, String culoare) {
        super(nume, consum);
        this.intensitate = intensitate;
        this.culoare = culoare;
    }
    @Override
    public void control() {
        stare = !stare;
        System.out.println(nume + " a fost " + (stare ? "pornit" : "oprit"));
    }
    @Override
    public void activareModEco() {
        intensitate = Math.max(intensitate - 50, 0); // scade intensitatea cu 50
        consumEnergetic *= 0.7; // reducere consum cu 30%
        System.out.println(nume + " in mod Eco. Intensitate: " + intensitate +
            ", Consum: " + consumEnergetic + "W");
    }
    @Override
    public void resetare() {
        intensitate = 100;
        culoare = "neutru";
        System.out.println(nume + " resetat.");
    }
    @Override
    public void afiseazaInfo() {
        super.afiseazaInfo();
        System.out.println(" Intensitate: " + intensitate + ", Culoare: " + culoare);
    }
}
```

Întrebare 6: Rezolvare

```
class ComparatorDispozitive implements Comparator<Dispozitiv> {
    @Override
    public int compare(Dispozitiv d1, Dispozitiv d2) {
        // Mai intai dupa stare (pornite primele)
        if (d1.isPornit() != d2.isPornit()) {
            return -1; // d1 pornit, d2 oprit -> d1 primul
        }
        if (!d1.isPornit() & d2.isPornit()) {
            return 1; // d1 oprit, d2 pornit -> d2 primul
        }
        // Daca au aceeasi stare, sortam descrescator dupa consum energetic
        if (d2.getConsumEnergetic() > d1.getConsumEnergetic()) {
            return 1;
        }
        if (d2.getConsumEnergetic() < d1.getConsumEnergetic()) {
            return -1;
        }
        return 0;
    }
}

class UtilizareCasaSmart {
    // Calculeaza consumul DOAR pentru dispozitivele PORNITE
    public static double calculeazaConsumTotal(List<Dispozitiv> dispozitive) {
        double consumTotal = 0;
        for (Dispozitiv dispozitiv : dispozitive) {
            if (dispozitiv.isPornit()) {
                consumTotal += dispozitiv.getConsumEnergetic();
            }
        }
        return consumTotal;
    }
}
```

Întrebare 6: Rezolvare

```
public class Main {
    public static void main(String[] args) {
        // Creare lista de dispozitive
        ArrayList<Dispozitiv> dispozitive = new ArrayList<>();
        dispozitive.add(new Televizor("Televizor Samsung", 150, 5, 20));
        dispozitive.add(new Frigider("Frigider LG", 200, 4, 300));
        dispozitive.add(new Bec("Bec Philips", 10, 80, "alb rece"));
        dispozitive.add(new Televizor("Televizor Sony", 250, 3, 15));
        dispozitive.add(new Bec("Bec Living", 15, 100, "alb cald"));

        // Afisare total dispozitive
        System.out.println("CREARE DISPOZITIVE");
        Dispozitiv.afiseazaTotalDispozitive();

        // Afisare informatii despre fiecare dispozitiv
        System.out.println("\n INFORMATII DISPOZITIVE");
        for (Dispozitiv dispozitiv : dispozitive) {
            dispozitiv.afiseazaInfo();
            System.out.println();
        }

        // Pornire dispozitive (toate)
        System.out.println(" PORNIRE DISPOZITIVE");
        for (Dispozitiv dispozitiv : dispozitive) {
            dispozitiv.control();
        }

        // Oprire anumite dispozitive
        System.out.println("\nOPRIRE SELECTIVA");
        dispozitive.get(1).control(); // Oprire Frigider
        dispozitive.get(3).control(); // Oprire Televizor Sony
    }
}
```


Întrebare 6: Rezolvare

```
// Calculare consum total (doar dispozitive pornite)
System.out.println("\nCONSUM TOTAL ");
double consumTotal = UtilitareCasaSmart.calculeazaConsumTotal(dispozitive);
System.out.println("Consum total dispozitive pornite: " + consumTotal + "W");

// Activare mod Eco pentru toate dispozitivele
System.out.println("\n ACTIVARE MOD ECO ");
for (Dispozitiv dispozitiv : dispozitive) {
    if (dispozitiv instanceof SmartControl) {
        ((SmartControl) dispozitiv).activareModEco();
    }
}

// Recalculare consum dupa mod Eco
System.out.println("\n CONSUM DUPA MOD ECO");
consumTotal = UtilitareCasaSmart.calculeazaConsumTotal(dispozitive);
System.out.println("Consum total dupa Eco: " + consumTotal + "W");

// Sortare dupa stare si consum
System.out.println("\n SORTARE DISPOZITIVE");
UtilitareCasaSmart.sorteazaDupaStareSiConsum(dispozitive);

System.out.println("Dispozitive sortate (pornite primele, descrescator dupa consum):"
);
for (Dispozitiv dispozitiv : dispozitive) {
    System.out.println(dispozitiv);
}
}
```

UML Diagram:

